

Preguntas

Preguntas sobre Reinforcement Learning

Guía de preguntas y ejercicios prácticos

1. Preguntas sobre Gymnasium

1. ¿Cuáles son los 5 retornos de `env.step()`?

```
observation, reward, terminated, truncated, info = env.step(action)
```

Enumerá y explicá brevemente el propósito de cada uno

- **observation:** el **nuevo estado** tras aplicar la acción, con la forma que define **observation_space** (ej. (pos, vel) en MountainCar, la matriz de frames en el DQN de highway-env del Obligatorio).
- **reward:** recompensa escalar obtenida en ese paso.
- **terminated:** True si el episodio **terminó por una razón natural** que el Proceso de Decisión de Markov (Markov Decision Process - MDP) llegó a estado terminal: el auto llega a la bandera, la mano de Blackjack se resuelve, caés al agua en FrozenLake.
- **truncated:** True si el **episodio se cortó por algo externo** al MDP (típicamente un límite de pasos impuesto por el wrapper(*) **TimeLimit**), no porque el agente llegó a un final real.
- **info:** dict con **datos auxiliares/diagnóstico**. No debería usarse para decidir la política, solo para debug o logging.

La distinción **terminated vs truncated** importa para el bootstrapping: en Q-learning/Sarsa no deberías bootstrapear con $\max_a Q(s',a)$ si el estado es terminal real (ver el * (not terminated) que usan en **q_learning_frozen** del lab de TD), pero sí seguís bootstrapeando si fue un corte por truncamiento.

2. ¿Qué información contiene `env.observation_space` cuando es un Box?

```
print(env.observation_space)
# Ej: Box(low=array(...), high=array(...), shape=(4,), dtype=float32)
```

¿Qué nos dice `.low`, `.high`, `.shape` y `.dtype`?

- **.low / .high:** cotas mínima/máxima de cada componente de la observación (se usan literalmente en `discretize_obs` del lab de TD: `env_low`, `env_high`).

- **.shape:** la forma del vector/tensor de observación (ej. (2,) en MountainCar: posición y velocidad; (4, 84, 336) en el highway-env del Obligatorio tras el pipeline de wrappers).
- **.dtype:** tipo numérico (float32 para observaciones continuas, uint8 para frames de imagen antes de normalizar).

3. ¿Qué ocurre si hacés env.step() tras un paso con terminated=True sin llamar a reset()?

No está prohibido a nivel de código (no tira excepción en la mayoría de los ambientes), pero es comportamiento no definido por la API: cada entorno hace algo distinto y en general no tiene sentido físico (seguir "jugando" una mano de Blackjack ya resuelta, seguir moviendo un péndulo que ya se cayó).

Los valores de **reward/observation** que devuelva no son confiables para entrenar. La documentación de Gymnasium es explícita: hay que llamar a **reset()** apenas **terminated** or **truncated** es True, antes de seguir llamando a **step()**.

4. ¿Qué devuelve cada llamada a env.reset()?

```
a, b = env.reset(seed=42)
```

¿Qué representan cada uno (a y b)?

- **a = observation:** no es cualquier estado — es un **estado inicial válido para ese entorno**, generado según su lógica interna de inicialización (ej. en Blackjack se reparte una mano nueva; en highway-env se spawnen los vehículos; en MountainCar el auto arranca en una posición aleatoria dentro de un rango).
Además, reset() reinicia por completo el estado interno del entorno — no solo devuelve una observación nueva, sino que "olvida" todo lo que pasó en el episodio anterior.
- **b = info:** en la práctica, para muchos entornos simples (CartPole, MountainCar) **suele venir como un diccionario vacío {}**. En otros trae datos de diagnóstico específicos del entorno (por ejemplo, en Blackjack puede indicar detalles de la config sab; en highway-env puede traer métricas como velocidad o si hubo colisión).
Es auxiliar/debug — no debería usarse para tomar decisiones de política, igual que el info de step().

5. ¿Para qué sirve pasar seed a env.reset(seed=...)?

Explicá su utilidad.

Fija la semilla del generador de números aleatorios interno del entorno (**np.random**). Con eso,

cualquier componente estocástico del entorno (el dealer en Blackjack, el hielo resbaladizo en FrozenLake `is_slippery=True`, el estado inicial) se vuelve reproducible.

Lo usan en el Obligatorio (`env.reset(seed=SEED)`, `make_env(..., seed=SEED)`) justamente para poder comparar DQN vs Double DQN en igualdad de condiciones y para que los resultados sean reproducibles.

6. Al crear el ambiente, ¿qué efecto tiene el parámetro `render_mode`?

```
env = gymnasium.make("CartPole-v1", render_mode="rgb_array")
```

- **"rgb_array"**: `env.render()` devuelve un array numpy (frame RGB). Es el modo que usan en todos los labs para poder envolver el entorno con `RecordVideo` y grabar episodios (MountainCar, CliffWalking, FrozenLake, Actor-Critic, highway-env).
- **"human"**: abre una ventana y renderiza en vivo.

Si no se especifica `render_mode`, `render()` típicamente no hace nada útil (puede devolver `None`).

7. Verdadero o Falso

"Gymnasium solo permite usar ambientes incluidos en su librería."

Falso. Se pueden registrar y usar ambientes custom.

- **GridWorldEnv** y **KBanditsEnv** son entornos 100% custom escritos para los labs.
- **highway-env** (Obligatorio) es una librería externa que se registra en el registry de Gymnasium y se usa con `gym.make(ENV_NAME, ...)` exactamente igual que un entorno nativo.

8. ¿Para qué sirven los wrappers en Gymnasium?

- Escribir dos ejemplos reales que usaron.
- ¿Qué hace cada uno?

Qué es un Wrapper?

Un wrapper en Gymnasium es una clase que envuelve un env existente para modificar o interceptar su comportamiento (`step`, `reset`, `render`, los espacios de observación/acción), sin tener que tocar el código del entorno original. Es el patrón de diseño decorator: el wrapper tiene la misma interfaz que un env normal (podés seguir llamando `.step()`, `.reset()`), pero por dentro delega al entorno envuelto y le agrega o cambia algo en el medio.

- **Por qué se usa:** Te permite componer transformaciones de forma modular: en vez de reescribir el entorno cada vez que necesitás algo distinto (grabar video, cambiar la observación, limitar pasos), apilas wrappers uno sobre otro. Cada wrapper hace una sola cosa y son reutilizables entre proyectos.

Ejemplos que aparecen en las notebooks:

1. **RecordVideo** (lab de TD, Actor-Critic, Obligatorio): graba video de episodios completos a disco, con `episode_trigger` controlando cada cuántos episodios grabar. No modifica la dinámica del entorno, solo intercepta `render()`.
2. **TransformObservation** (lab de TD): transforma la observación cruda aplicando una función arbitraria.
Lo usan para discretizar MountainCar (continuo → MultiDiscrete([NUMBER_STATES, NUMBER_STATES])) y así poder usar Q-learning/Sarsa tabular.

Bonus del Obligatorio (pipeline completo de wrappers estilo Mnih et al. 2013 sobre highway-env): un wrapper custom **_RenderAsObservation** (reemplaza la observación nativa por el frame RGB), **GrayscaleObservation** (a escala de grises con luma BT.601) y **FrameStackObservation** (apila 4 frames para que la red pueda inferir velocidad/aceleración por diferencia entre frames).

9. ¿Qué error hay en este bucle de juego?

```
obs, info = env.reset()

for _ in range(500):
    obs, reward, terminated, truncated, info = env.step(
        env.action_space.sample()
    )

    if terminated:
        obs, info = env.reset()
```

✗ El error:

solo resetea cuando `terminated` es `True`, pero ignora `truncated`. Si el episodio se corta por un **TimeLimit** (`truncated=True, terminated=False`), el bucle sigue llamando `step()` sobre un entorno que ya terminó, sin resetear.

✓ Corrección:

```
if terminated or truncated:
    obs, info = env.reset()
```

10. ¿Cómo limitarías manualmente un episodio a 50 pasos si el ambiente no tiene TimeLimit?

Dos formas:

a) Contando pasos a mano y forzando el corte:

```
obs, info = env.reset()
for step in range(50):
    action = env.action_space.sample()
    obs, reward, terminated, truncated, info = env.step(action)
    if step == 49:
        truncated = True
    if terminated or truncated:
        break
```

b) Forma "nativa" de Gymnasium — envolver el entorno con el wrapper que ya existe para esto:

```
from gymnasium.wrappers import TimeLimit

env = TimeLimit(env, max_episode_steps=50)
```

Este wrapper cuenta los pasos internamente y devuelve **truncated=True** automáticamente al llegar a 50, sin que tengas que llevar el contador vos mismo.

2. Preguntas prácticas sobre K-Bandits

1. Selección de acción greedy

Supongamos que tenés un array `Q` de tamaño `K` (valores estimados).

¿Cómo seleccionarías la acción greedy en código NumPy?

`action = np.argmax(Q)`

Devuelve el índice del brazo con mayor valor estimado. Ojo con los empates: **`np.argmax`** siempre devuelve el primer máximo, lo que sesga sistemáticamente hacia acciones de índice bajo cuando hay valores iguales (por eso en el lab de TD usan **`random_argmax`**, que desempata al azar entre los máximos).

2. Implementación de ϵ -greedy (pseudocódigo)

Escribí un fragmento de código o pseudocódigo para elegir una acción bajo la política ϵ -greedy, usando:

- `Q`
- `epsilon`
- `np.random.rand()`

Tal cual lo implementan en **`EpsilonGreedyAgent.select_action`**:

```
if np.random.rand() < epsilon:
    action = np.random.randint(len(Q))    # explorar: acción uniforme al azar
else:
    action = np.argmax(Q)                 # explotar: la mejor acción estimada
```

Con probabilidad `epsilon` explorás (acción uniforme, sin importar `Q`), y con probabilidad `1 - epsilon` explotás la acción greedy actual.

3. Cálculo de ϵ annealado

Dado:

- `epsilon_start`
- `epsilon_end`
- `epsilon_anneal`
- `t` (pasos actuales)

Escribí el código o pseudocódigo para calcular el valor de `epsilon` en el paso `t`.

Mismo patrón que usan en **`abstract_agent.py`** (`compute_epsilon`), aplicado acá con los nombres de la pregunta — anneal lineal:

```
def compute_epsilon(epsilon_start, epsilon_end, epsilon_anneal, t):
    if t < epsilon_anneal:
        return epsilon_start - (epsilon_start - epsilon_end) * (t / epsilon_anneal)
    return epsilon_end
```

Es una interpolación lineal: en $t=0$ vale `epsilon_start`, decrece linealmente, y a partir de $t=\text{epsilon_anneal}$ queda clamped en `epsilon_end` (nunca sigue bajando). Esto asegura que el agente explore mucho al principio y converja a explotación casi total (pero nunca cero, salvo que `epsilon_end=0`) a largo plazo.

4. Comentario de error en actualización de Q

```
q[arm] = reward
```

Comentá por qué esta línea está mal para aprendizaje incremental de Q, y qué implementación se usaría en su lugar.

Esta línea sobrescribe la estimación anterior de $Q[\text{arm}]$ con el reward de un solo paso, en vez de promediarlo con la historia de recompensas observadas para ese brazo. Perdés toda la información acumulada: si el brazo dio recompensas $[1, 1, 1, 1]$ y ahora sale un 0 por ruido, $Q[\text{arm}]$ pasa de 1 a 0 de golpe, en vez de moverse levemente hacia 0.

Lo correcto es una actualización incremental (la que implementan en `EpsilonGreedyAgent.update_internal_state`):

```
n_a[arm] += 1
q_a[arm] += (reward - q_a[arm]) / n_a[arm]
```

Esto es matemáticamente equivalente al promedio de todos los rewards observados hasta ahora para ese brazo, pero sin tener que guardarlos todos.

5. Uso del array N

En lugar de guardar todos los rewards para cada brazo (una lista), usamos un array N (veces que se eligió cada brazo).

¿Por qué? ¿Cuál es su utilidad en el cálculo del nuevo $Q[\text{arm}]$?

Escribir la actualización:

```
Q[arm] = ...
```

En vez de guardar una lista de todos los rewards por brazo (que crece sin límite en memoria y obliga a recalcular el promedio de cero cada vez), guardamos solo el contador $N[\text{arm}]$ = cantidad de veces que se eligió ese brazo. Con eso alcanza para hacer la actualización incremental exacta del promedio muestral (regla de Sutton & Barto, ecuación 2.3):

$$Q[\text{arm}] = Q[\text{arm}] + (\text{reward} - Q[\text{arm}]) / N[\text{arm}]$$

$N[\text{arm}]$ hace de "tasa de aprendizaje decreciente" $1/N[\text{arm}]$: al principio (N chico) cada nuevo reward mueve mucho a Q , y a medida que N crece cada reward pesa cada vez menos — lo cual converge al promedio real de la distribución de recompensas de ese brazo (asumiendo que es estacionaria).

6. ¿Qué pasa si usamos un α constante?

Reescribí la actualización anterior para usar una tasa de aprendizaje constante α en lugar de $1/N[\text{arm}]$.

¿Cómo se modificaría la línea de actualización?

$$Q[\text{arm}] = Q[\text{arm}] + \alpha * (\text{reward} - Q[\text{arm}])$$

(exactamente lo que hace `OptimisticEpsilonGreedyAgent.update_internal_state` con `self.alpha`). A diferencia de $1/N[\text{arm}]$, acá el peso que le das al reward más reciente no decrece con el tiempo — queda fijo en α . Esto hace que Q sea un promedio exponencialmente ponderado, dando más peso a las observaciones recientes y menos a las viejas. Es preferible en entornos no estacionarios (donde el valor real de un brazo puede cambiar con el tiempo), porque $1/N$ haría que el agente "confíe" cada vez más en un promedio histórico que ya no representa la realidad actual del brazo.

7. Inicialización optimista

Si en lugar de

```
Q = np.zeros(K)
```

usamos

```
Q = np.ones(K) * optimistic_value
```

¿Qué efecto práctico tiene en la exploración?

```
Q = np.ones(K) * optimistic_value # optimistic_value alto, ej. mayor a cualquier reward  
esperado
```

Efecto práctico:

fuerza exploración temprana automática, sin necesidad de un epsilon alto. Como todos los brazos arrancan con un valor sobreestimado, la primera vez que probás cualquier brazo el reward real (más bajo que la estimación inicial) hace bajar su Q, así que la próxima acción greedy va a preferir otro brazo (el que todavía no "depcionó"). El agente termina probando todos los brazos al menos una vez antes de asentarse en explotación pura — es la técnica que en el lab llaman "optimistic initial values" (Sutton & Barto, sección 2.6)

En OptimisticEpsilonGreedyAgent la combinan con alpha constante en vez de $1/N$, porque con $1/N$ el efecto del optimismo se diluiría muy rápido (el primer update ya divide por $N=1$, dejando poco rastro del valor inicial en las siguientes iteraciones).

3. Programación Dinámica

En la notebook representamos la política como:

```
pi = {
    (state, a): probabilidad
}
```

Si el espacio de acciones es discreto (`n_actions`), escribí un fragmento de código para muestrear una acción en base a las probabilidades de la política, dado un estado `s1`.

```
def select_action(state, pi, n_actions):
    """
    Dado un estado y una política pi = {(state, a): probabilidad},
    muestrea una acción según la distribución de pi para ese estado.
    """
    probs = [pi.get((state, a), 0) for a in range(n_actions)]

    return np.random.choice(n_actions, p=probs)

accion = select_action(s1, pi, n_actions)
```

Por qué es así:

- 1. `pi.get((state, a), 0)`:** como `pi` es un diccionario disperso (solo tiene entradas para los pares `(s,a)` que realmente le asignaron probabilidad), usar `.get(..., 0)` evita un `KeyError` si algún par no está — se asume probabilidad 0 por defecto.
- 2. `probs`** debe sumar 1 para ese estado — es responsabilidad de quien construye `pi` (como en `improve_policy/value_iteration`, que reparten `1/len(best_actions)` entre las acciones óptimas empatadas).
- 3. `np.random.choice(n_actions, p=probs)`:** muestrea un índice de acción entre 0 y `n_actions-1` respetando esas probabilidades. Si la política es determinista (una sola acción con `prob=1`), siempre devuelve esa acción; si es estocástica (como en los casos de empate de Q-values del `improve_policy`), samplea entre las acciones óptimas.

Cuando hay empate en el Q-value máximo entre varias acciones, la política óptima resultante no es única — cualquier distribución de probabilidad sobre ese subconjunto de acciones empatadas sigue siendo óptima. `improve_policy` en el lab elige repartir uniformemente (`1/len(best_actions)`), pero podría haberse quedado con una sola de ellas (determinista) y seguiría siendo óptima.

4. Monte Carlo

Dado el siguiente código:

```
def monte_carlo_prediction(policy, env, number_episodes, gamma=1.0):

    V = defaultdict(float)
    returns = defaultdict(lambda: [])

    for _ in range(number_episodes):

        episode = generate_episode(policy, env)
        T = len(episode)
        t = T
        G = 0

        states = [state for state, _, _ in episode]

        for state, _, reward, _ in episode[::-1]:
            t -= 1
            G = (gamma * G) + reward

            if state not in states[:t]:
                returns[state].append(G)
                V[state] = np.mean(returns[state])

    return V
```

a) ¿El código anterior implementa First-Visit o Every-Visit?

Justifícalo.

Es First-Visit. La clave está en esta línea, dentro del recorrido hacia atrás del episodio:

```
for state, _, reward, _ in episode[::-1]:
    t -= 1
    G = (gamma * G) + reward
    if not state in states[:t]:
        returns[state].append(G)
        V[state] = np.mean(returns[state])
```

Como se itera `episode[::-1]` (de atrás para adelante), `t` arranca en `T` y se decrementa en cada paso, así que en cada iteración `t` coincide con el índice forward (cronológico) del estado que se está procesando en ese momento. `states[:t]` son entonces todos los estados que ocurrieron antes de ese instante en el orden original del episodio.

La condición `if state not in states[:t]` solo es verdadera cuando el estado actual no volvió a aparecer antes en el episodio — es decir, cuando el timestep que se está procesando es la primera vez (cronológicamente) que se visita ese estado. Si el mismo estado aparece más de una vez en el episodio, sólo se registra el retorno `G` correspondiente a su primera ocurrencia; las visitas posteriores a ese mismo estado se descartan (la condición da `False` porque el estado sí aparece antes en `states[:t]`).

Este es exactamente el mismo patrón que usan ustedes en `mc_onpolicy_control` del lab de Monte Carlo (`if (state, action) not in sa_pairs[:t]: # first-visit`), así que es consistente con la convención del curso.

b) ¿Cómo lo modificarías para implementar el otro caso?

Basta con eliminar el chequeo de "primera visita" y acumular el retorno en cada aparición del estado, sin filtrar:

```
def monte_carlo_prediction_every_visit(policy, env, number_episodes, gamma=1.0):
    V = defaultdict(float)
    returns = defaultdict(lambda: [])

    for _ in range(number_episodes):
        episode = generate_episode(policy, env)
        G = 0

        for state, _, reward, _ in episode[::-1]:
            t += 1
            G = (gamma * G) + reward
            if not state in states[:t]:
            returns[state].append(G)
            V[state] = np.mean(returns[state])

    return V
```

Ya no hace falta ni `T`, ni `t`, ni la lista `states` — esas variables existían únicamente para poder hacer el chequeo de primera visita. En Every-Visit, cada vez que el estado aparece en el episodio se trata como una observación independiente de retorno, así que se promedian todas las apariciones de todos los episodios, no solo la primera de cada uno.

5. Temporal Difference

1. En MountainCar, ¿por qué era necesario discretizar las observaciones antes de usarlas para los algoritmos TD?

Los algoritmos tabulares que vieron en el curso (Q-learning, Sarsa) representan Q como una tabla indexada por (estado, acción) — literalmente un array de NumPy con un índice entero por cada estado posible ($Q = \text{np.zeros}((\text{NUMBER_STATES}, \text{NUMBER_STATES}, \text{n_actions}))$). En MountainCar, la observación es continua (Box: posición y velocidad, ambas float32), así que hay infinitos estados posibles — no se puede indexar un array con un float. Discretizar (bucketizar posición y velocidad en NUMBER_STATES intervalos) convierte el espacio de estados continuo en uno finito y discreto, lo que permite usar una tabla $Q[\text{pos}, \text{vel}, \text{action}]$ en vez de necesitar un aproximador de función (como una red neuronal, que es lo que sí se usa más adelante con DQN en el Obligatorio).

2. ¿En qué rango de valores queda cada dimensión discretizada usando el siguiente código si NUMBER_STATES = 20?

```
def discretize_obs(obs):  
  
    env_low = env.observation_space.low  
    env_high = env.observation_space.high  
  
    env_den = (env_high - env_low) / NUMBER_STATES  
  
    pos_den = env_den[0]  
    vel_den = env_den[1]  
  
    pos_low = env_low[0]  
    vel_low = env_low[1]  
  
    pos_scaled = int((obs[0] - pos_low) / pos_den)  
    vel_scaled = int((obs[1] - vel_low) / vel_den)  
  
    return pos_scaled, vel_scaled
```

Con el código:

```
env_den = (env_high - env_low) / NUMBER_STATES  
  
pos_scaled = int((obs[0] - pos_low) / pos_den)  
vel_scaled = int((obs[1] - vel_low) / vel_den)
```

Cada dimensión se mapea, en teoría, al rango de índices enteros $\{0, 1, \dots, 19\}$ (20 buckets de igual ancho): cuando obs está en su valor mínimo ($\text{pos_low}/\text{vel_low}$), el índice da 0; a medida que obs crece, el índice sube hasta acercarse a $\text{NUMBER_STATES} - 1 = 19$.

Ojo con el caso borde: si $\text{obs}[0]$ llega exactamente a pos_high (el máximo del observation_space , ej. 0.6 en MountainCar), la cuenta da $(\text{pos_high} - \text{pos_low})/\text{pos_den} = \text{NUMBER_STATES} = 20$ exacto, y $\text{int}(20) = 20$ — un índice fuera de rango para un array de tamaño 20 (índices válidos 0..19). En la práctica es poco frecuente porque requiere que la observación caiga justo en el límite exacto, pero es un off-by-one latente en el $\text{int}()$ truncado sin clip. Una versión más robusta agregaría $\text{min}(\text{pos_scaled}, \text{NUMBER_STATES} - 1)$.

3. Para implementar la actualización de Q-learning:

$$Q(S,A) \leftarrow Q(S,A) + \alpha [R + \gamma \cdot \max_a Q(S',a) - Q(S,A)]$$

Supongamos que:

```
Q = np.zeros(
    (
        NUMBER_STATES,
        NUMBER_STATES,
        env.action_space.n
    )
)
```

¿Cómo obtenés la máxima acción ($\max Q(S',a)$) en código para un par de índices (pos, vel) del estado siguiente?

Escribir el fragmento completo para la actualización de Q-learning dentro de un ciclo de entrenamiento usando NumPy.

```
for _ in tqdm(range(num_episodes)):

    (pos, vel), _ = env.reset()
    done = False
    episode_reward = 0

    while not done:

        # Epsilon-greedy
        if np.random.rand() < epsilon:
            action = env.action_space.sample()
        else:
            action = np.argmax(Q[pos, vel])

        # Take action
        (next_pos, next_vel), reward, terminated, truncated, info = env.step(action)
        episode_reward += reward

        # Update Q-value
        best_next_action = ...
        Q[pos, vel, action] += ...

        # Update state
        (pos, vel) = (next_pos, next_vel)

    done = terminated or truncated
```

Usando la tabla $Q = \text{np.zeros}((\text{NUMBER_STATES}, \text{NUMBER_STATES}, \text{env.action_space.n}))$ y siguiendo exactamente el patrón que implementan en `q_learning()` del lab:

```

for _ in tqdm(range(num_episodes)):
    (pos, vel), _ = env.reset()

    done = False

    episode_reward = 0

    while not done:

        # Epsilon-greedy action selection
        if np.random.rand() < epsilon:
            action = env.action_space.sample() # Explore
        else:
            action = np.argmax(Q[pos, vel])    # Exploit

        # Take action
        (next_pos, next_vel), reward, terminated, truncated, info = env.step(action)
        episode_reward += reward

        # Update Q-value
        best_next_action = np.max(Q[next_pos, next_vel])
        td_target = reward + gamma * best_next_action
        Q[pos, vel, action] += alpha * (td_target - Q[pos, vel, action])

        # Update state
        (pos, vel) = (next_pos, next_vel)
        done = terminated or truncated

```

$$Q(S,A) \leftarrow Q(S,A) + \alpha [R + \gamma \cdot \max_a Q(S',a) - Q(S,A)]$$

Puntos clave:

- **np.max(Q[next_pos, next_vel]):** Q[next_pos, next_vel] indexa toda la fila de acciones para ese estado siguiente (un array de tamaño n_actions), y np.max toma directamente el valor máximo — no hace falta argmax porque el update de Q-learning solo necesita el valor max_a Q(S', a), no la acción que lo logra (a diferencia de Sarsa, que sí necesita la acción realmente elegida A' para el siguiente Q(S', A')).

- **Q[pos, vel, action] += alpha * (td_target - Q[pos, vel, action]):** es la regla TD del enunciado, $Q(S,A) \leftarrow Q(S,A) + \alpha [R + \gamma \cdot \max_a Q(S',a) - Q(S,A)]$, escrita como actualización incremental in-place.

- **Es off-policy:** la acción que se ejecutó (action, elegida ϵ -greedy) puede ser distinta de la acción usada para el target (argmax/max sobre Q[next_pos, next_vel], que es greedy pura) — esa es la diferencia central con Sarsa, que sí es on-policy y usa la acción A' efectivamente seleccionada por la política de comportamiento.

6. PyTorch

1. Tensor vs NumPy array

¿En qué se diferencian un `torch.Tensor` y un `np.ndarray`?

Ambos representan arrays n-dimensionales con operaciones vectorizadas similares (de hecho comparten mucha API), pero se diferencian en:

- **Autograd:** un `torch.Tensor` puede llevar registro del grafo computacional (`requires_grad=True`) para calcular gradientes automáticamente con `.backward()`. Un `np.ndarray` no tiene ningún mecanismo de diferenciación automática.
- **Dispositivo (device):** un tensor puede vivir en CPU, GPU (cuda) o MPS (Apple Silicon) y las operaciones se ejecutan ahí, aprovechando paralelismo masivo. NumPy solo corre en CPU.
- **Integración con nn.Module/optimizadores:** los tensores son el tipo de dato nativo que consumen las capas de PyTorch (`nn.Linear`, `nn.Conv2d`, etc.) y los optimizadores (`torch.optim`).

Conversión entre ambos es directa (`torch.from_numpy(arr)`, `tensor.numpy()`), y comparten memoria cuando están en CPU — como hacen en `process_state` del Obligatorio (`torch.from_numpy(arr).float().div_(255.0)`).

2. Propiedades de Tensor

Listá y describí brevemente las propiedades de un tensor:

- **dtype:** tipo de dato de los elementos (`torch.float32`, `torch.uint8`, `torch.bool`, etc.). Importa para memoria y precisión — ej. en el Obligatorio las observaciones viajan como `uint8` [0,255] y recién se convierten a `float32` justo antes de entrar a la red, para no inflar el replay buffer.
- **shape (o .size()):** la forma/dimensiones del tensor. Ej. (4, 84, 336) para el stack de 4 frames en escala de grises del highway-env.
- **ndim (o .dim()):** cantidad de dimensiones del tensor (ej. `ndim=4` para un batch de imágenes (Batch, Channels, Height, Weight)).
- **device:** dónde vive físicamente el tensor (`cpu`, `cuda`, `mps`). Las operaciones entre tensores requieren que todos estén en el mismo device — de ahí el patrón `images = images.to(DEVICE)` al inicio de cada batch.
- **requires_grad:** si es `True`, PyTorch registra las operaciones sobre ese tensor en el grafo computacional para poder calcular dL/dx con `.backward()`. Los pesos de un modelo (`nn.Parameter`) lo tienen en `True` por defecto; los datos de entrada normalmente no lo necesitan.

3. Argumentos clave de nn.Linear

```
nn.Linear(in_features, out_features)
```

¿Qué representa cada argumento y cómo define el comportamiento de la capa?

Como en FeedForwardModel del lab (`self.linear1 = nn.Linear(in_features=784, out_features=128)`):

- **in_features: dimensión del vector de entrada** que recibe la capa (**cuántas neuronas/features** tiene la capa anterior, o el input aplanado — $784 = 28 \times 28$ para una imagen de MNIST/FashionMNIST).
- **out_features: cuántas neuronas** tiene esta capa, es decir, la **dimensión del vector de salida**.

Internamente, la capa aplica $y = xW^T + b$, con W de forma $(out_features, in_features)$ y b de forma $(out_features,)$ — ambos son parámetros entrenables. Estos dos argumentos fijan el tamaño de esa matriz de pesos y, por lo tanto, cuántos parámetros tiene la capa ($in_features \times out_features + out_features$).

4. Argumentos clave de nn.Conv2d

```
nn.Conv2d(  
    in_channels,  
    out_channels,  
    kernel_size,  
    stride=1,  
    padding=0  
)
```

Explicá el rol de cada argumento.

Como en ConvolutionalModel (`nn.Conv2d(in_channels=1, out_channels=16, kernel_size=3, padding=1)`) y en DQN_CNN_Model del Obligatorio (que recibe 4 canales, uno por frame apilado):

- **in_channels:** cantidad de **canales** de la entrada (1 para escala de grises, 3 para RGB, o NUM_STACKED_FRAMES como en el DQN del Obligatorio donde cada "canal" es un frame temporal distinto).

- **out_channels:** cantidad de **filtros/kernels** que aplica la capa — cada uno produce un **feature map de salida**, así que también es la cantidad de canales de la salida.
- **kernel_size:** **tamaño espacial del filtro convolucional** (ej. 3 → kernel de 3×3).
- **stride:** **cuántos píxeles se desplaza el kernel** entre aplicaciones consecutivas. stride=1 (default) recorre pixel a pixel; valores mayores reducen la resolución espacial de salida (downsampling), como se usa típicamente en las primeras capas de una CNN estilo DQN de Atari.
- **padding:** **cuántos píxeles de relleno (ceros) se agregan en los bordes de la entrada** antes de convolucionar. Con padding=1 y kernel_size=3, stride=1, la salida conserva el mismo alto/ancho que la entrada (como comentan en ConvolutionalModel: entra 28×28, sale 28×28 antes del pooling).

5. Código de entrenamiento: ¿qué hace cada línea?

```
images = images.to(DEVICE)
labels = labels.to(DEVICE)

optimizer.zero_grad()

predictions = model(images)

loss = loss_func(predictions, labels)

loss.backward()

optimizer.step()
```

Explicá brevemente qué hace cada línea y por qué es necesaria en un ciclo de entrenamiento.

- 1. images = images.to(DEVICE) / labels = labels.to(DEVICE):** mueve el batch de datos al mismo dispositivo (CPU/GPU) donde vive el modelo. Es obligatorio: PyTorch no puede operar entre tensores en devices distintos.
- 2. optimizer.zero_grad():** **pone en cero los gradientes acumulados** en .grad de todos los parámetros del modelo. Es necesario porque PyTorch acumula gradientes en cada .backward() por defecto — si no se resetean, se sumarían los del batch anterior con los del actual, corrompiendo la actualización.
- 3. predictions = model(images):** forward pass — **pasa el batch por la red y obtiene las predicciones** (logits, en el caso de clasificación).
- 4. loss = loss_func(predictions, labels):** **calcula el valor escalar de la función de costo** comparando predicciones vs. etiquetas verdaderas (ej. cross-entropy).

5. `loss.backward()`: backward pass — usa autograd para calcular, vía regla de la cadena, el gradiente de loss respecto a todos los parámetros del modelo que tengan `requires_grad=True`, y los guarda en `param.grad`.

6. `optimizer.step()`: usa esos gradientes para actualizar los pesos del modelo según la regla del optimizador elegido (ej. SGD: `param -= lr * param.grad`; Adam aplica además momentum y normalización adaptativa). Sin este paso, calcular el gradiente no serviría de nada — es el que efectivamente "aprende".

Este es el ciclo estándar que se repite en cada batch, por cada época de entrenamiento, tanto en `Intro_to_Pytorch.ipynb` como (en su versión más compleja, con réplica de red target y muestreo de replay buffer) en el DQN/DDQN del Obligatorio.

Preguntas Extra

Parcial 2020

1. ¿Qué modificaciones fueron realizadas para adaptar el algoritmo de DeepMind para aplicarlo a los problemas del obligatorio?

El algoritmo de base es el DQN de Mnih et al. (2013), pensado para Atari. Highway-env (highway-fast-v0) es un entorno muy distinto en varios sentidos, así que hubo que adaptar varias piezas:

a) Forzar observación visual (pixel-based) donde el entorno no la da por defecto.

highway-env entrega nativamente una KinematicObservation: una matriz estructurada con posiciones/velocidades de los vehículos cercanos — un "oráculo simbólico" que resolvería el problema con un MLP chico en pocos miles de pasos, sin que el agente aprenda percepción. Se agregó un wrapper custom `_RenderAsObservation` que intercepta cada `step()` y reemplaza esa observación por el frame RGB (`env.render(mode="rgb_array")`), replicando el régimen de Atari (aprender política y percepción visual a la vez).

b) Pipeline de preprocesamiento adaptado a la geometría del nuevo entorno.

Mnih et al. resizean Atari (pantalla nativa 210×160) a 84×84 (cuadrado). El render nativo de highway es 600×150 (aspecto 4:1, escena tipo "autopista horizontal"), muy distinto de una pantalla de Atari. Se resizeó a 336×84 preservando ese aspect ratio 4:1 en vez de forzar un cuadrado, para no distorsionar la geometría de carriles/vehículos (a costa de una primera capa fully-connected más grande). Se mantuvo la conversión a escala de grises (BT.601) y el frame stacking (4 frames) del paper original, porque siguen siendo necesarios para inferir velocidad/dirección por diferencia entre frames.

c) Frame-skip implementado distinto (config del entorno, no wrapper externo).

Atari usa el wrapper `AtariPreprocessing(frame_skip=k)`. Highway-env no tiene ese wrapper; el mismo efecto (repetir la acción durante varios sub-pasos físicos y devolver solo el frame final) se logra ajustando la relación `simulation_frequency / policy_frequency` en la config del entorno — algorítmicamente idéntico, pero implementado a nivel de configuración del simulador en lugar de un wrapper de Gymnasium.

d) Espacio de acciones distinto.

Atari tiene acciones de joystick discretas y "crudas". highway-fast-v0 usa DiscreteMetaAction: 5 meta-acciones de alto nivel (cambiar de carril izq/der, mantener, acelerar, frenar), cada una ejecutada durante varios sub-pasos de física. No hubo que tocar la arquitectura de salida del DQN por esto (sigue siendo un Linear con n_actions salidas), pero sí cambia la semántica de qué representa cada acción.

e) Arquitectura de la red simplificada respecto al paper.

En vez de las 3 capas convolucionales + FC de 512 del DQN "completo", en dqn_cnn_model.py se sugiere una versión más chica (2 capas conv — 16@8×8/stride4, 32@4×4/stride2 — y una FC de 256), justificada por las restricciones de tiempo/cómputo de un entorno de curso (notebooks ORT / Colab T4, pocas horas por agente).

f) Hiperparámetros ajustados al presupuesto de cómputo disponible.

El replay buffer del paper original es de 1M transiciones; acá se usa BUFFER_SIZE = 10_000. TOTAL_STEPS = 100_000 (vs. los ~50M frames de Atari). Esto no es una diferencia "algorítmica" sino una adaptación práctica: con pocas horas de entrenam un horizonte de entrenamiento de esa escala serían inviables.

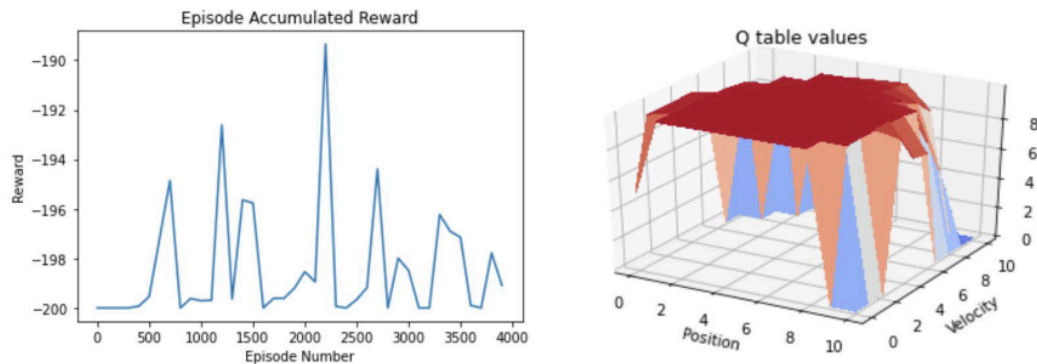
g) Reward shaping específico del entorno.

HIGH_SPEED_REWARD ajuste del entorno, no del algoritmo, para romper un óptimo local (el agente "igualando la velocidad del vecino" en que adelantar valga claramente más que ir crucero detrás de un auto lento).

h) Sin necesidad de reward clipping.

El paper de Atari clippea las magnitudes de recompensa, varían enormemente entre distintos juegos. En highway-env la recompensa ya viene definida para un único problema, así que no hizo falta ese clipping.

2. Considere las siguientes gráficas generadas por un estudiante intentando aplicar Q-Learning tabular al problema del Mountain Car:



- A. ¿Qué puede interpretar a partir de ambas gráficas? Justifique.
B. Claramente el agente obtenido no es el óptimo, ¿Cómo podría mejorarlo para llegar a una solución más óptima?
C. ¿Cómo esperaría que fueran las gráficas obtenidas para un agente mejor?

A. ¿Qué se puede interpretar de ambas gráficas?

Gráfica de reward por episodio: el reward se mantiene casi todo el entrenamiento (~4000 episodios) pegado al piso de -200 (el máximo castigo/timeout de MountainCar, ya que cada paso da -1 y el episodio corta a los 200 pasos si no se llega a la bandera), con picos aislados y ruidosos hacia valores algo mejores (-188 a -190). Lo importante es que no hay una tendencia de mejora sostenida: los picos buenos aparecen de forma dispersa a lo largo de todo el entrenamiento (incluso cerca del episodio 2200), no se van haciendo más frecuentes ni la línea de base sube con el tiempo. Esto sugiere que esos picos son producto de la exploración aleatoria (epsilon) llegando ocasionalmente a la bandera por suerte, no de una política aprendida que consistentemente resuelve el problema.

Gráfica de valores de la Q-table: se ve una superficie prácticamente plana en un valor alto (~8) sobre casi todo el rango de (position, velocity), con unos pocos "picos hacia abajo" angostos (valles azules que caen a 0) en combinaciones puntuales de estado. Esto no se parece en nada a la forma esperada de la función de valor óptima de MountainCar, que debería tener una estructura/gradiente suave: valores más altos en estados cercanos a la meta (posición+velocidad alineadas para alcanzar la bandera) y más bajos en estados lejos de ella, no un plateau casi uniforme con valles aislados.

Conclusión conjunta: ambas gráficas cuentan la misma historia — el agente no converge a una política útil. La meseta casi uniforme en la Q-table indica que la mayoría de los pares (estado, acción) nunca se visitaron lo suficiente como para diferenciar sus valores (quedaron cerca de su estimación inicial u obtuvieron muy pocos updates ruidosos), y los pocos "valles" corresponden a los raros estados que sí se visitaron bastante. Combinado con el reward estancado en -200, todo

apunta a que el agente casi nunca llegó a la bandera durante el entrenamiento, así que casi no hubo señal de reward informativa (-1 en todos lados salvo el objetivo) para diferenciar la Q-table.

B. El agente claramente no es óptimo — ¿cómo mejorarlo?

- **Más episodios/pasos de entrenamiento:** si la Q-table sigue casi plana a las 4000 episodios, probablemente no alcanzó ni para visitar bien el espacio de estados-acciones, y mucho menos para converger.

- **Revisar el schedule de ϵ :** si epsilon decae muy rápido, el agente deja de explorar antes de haber encontrado la bandera ni una sola vez y queda atrapado explotando una política mala; si decae muy lento (o el piso epsilon_min es muy alto), sigue siendo casi aleatorio todo el entrenamiento. Dado que los picos de reward están dispersos y no crecen en frecuencia, es más consistente con esto último.

- **Reward shaping:** el reward nativo de MountainCar es extremadamente disperso (-1 en todos lados salvo la meta), lo que hace muy difícil el credit assignment con Q-learning tabular puro. Agregar una recompensa moldeada (potential-based, ej. función de la posición/altura o de la energía potencial+cinética del auto) daría señal de aprendizaje mucho más densa y guiaría antes al agente hacia el comportamiento de "balancearse" para tomar impulso.

- **Discretización más fina o mejor diseñada:** si NUMBER_STATES es muy chico, muchos estados relevantes (sobre todo cerca de la meta o en zonas de velocidad crítica) podrían estar agrupados en muy pocos buckets, perdiendo la resolución necesaria para diferenciar acciones.

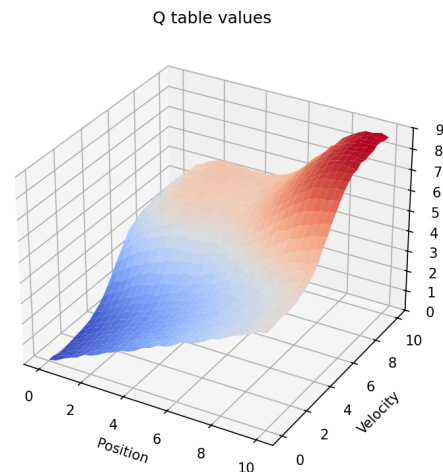
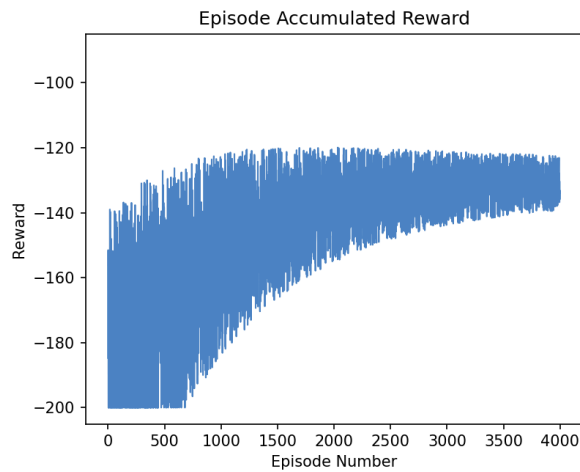
- **Ajustar alpha/gamma:** un gamma alto (cercano a 1) es importante porque el objetivo está lejos en el tiempo (se necesita "recordar" el valor de tomar impulso muchos pasos antes de llegar arriba); un alpha mal calibrado puede hacer que el aprendizaje sea demasiado ruidoso o demasiado lento.

- **Inicialización optimista (ver Parte 2 del cuestionario, pregunta 7):** en MountainCar el reward real es siempre negativo, así que inicializar Q en 0 ya es "optimista" respecto al valor real y debería empujar exploración — si aun así no converge, el problema probablemente está más en el schedule de exploración/duración del entrenamiento que en la inicialización.

C. ¿Cómo esperarías que fueran las gráficas para un agente mejor?

- **Reward por episodio:** debería arrancar cerca de -200 (al principio, con política aleatoria, casi nunca se llega a la bandera) y mostrar una tendencia claramente ascendente (menos negativa) a medida que avanza el entrenamiento, estabilizándose en una banda relativamente angosta y consistente — típicamente entre -100 y -160 para MountainCar tabular bien entrenado (episodios de ~110-160 pasos hasta alcanzar la bandera) —, con la varianza entre episodios disminuyendo conforme epsilon decae y la política se vuelve más determinística/consistente. No una nube de puntos dispersos entre -200 y -190 sin tendencia, como en la gráfica dada.

- **Q-table:** en vez del plateau casi uniforme con valles angostos, se esperaría una superficie con estructura suave y coherente: un gradiente que va subiendo hacia los estados donde posición y velocidad están "alineadas" para alcanzar la bandera (es decir, valores más altos en la dirección hacia la meta) y bajando hacia estados lejos de ella o con momentum en la dirección incorrecta — reflejando que la mayoría de los estados relevantes fueron visitados muchas veces y sus valores realmente convergieron, no que quedaron en su estimación inicial.



Parcial 2021

1. En el algoritmo de DeepMind para jugar Atari, se utilizan dos redes, una target network y una q network, **¿cómo manejo estas redes en su obligatorio?**

En el Obligatorio se implementan dos variantes con un manejo de redes diferente a propósito, para poder aislar el efecto de la target network en la comparación DQN vs. DDQN:

DQN (`dqn_agent.py`) — sin target network, fiel al paper de 2013

El docstring del propio archivo lo deja explícito: "Sin target network: el target Q se calcula con la propia policy_net". A diferencia del DQN "Nature" de 2015 (que sí introduce una segunda red), replicamos el paper original de Mnih et al. 2013 (arXiv:1312.5602), que todavía no usaba target network. Entonces acá hay una sola red (policy_net), que cumple dos roles a la vez:

- Selecciona la acción vía ϵ -greedy ($\arg\max_a Q(s,a)$).
- Calcula el target de Bellman para el update: $y = r + \gamma \cdot \max_a Q(s', a) \cdot (1 - \text{terminated})$, usando esa misma red que se está entrenando.

Esto es justamente lo que en el paper de 2015 se identifica como fuente de inestabilidad/sobreestimación (el objetivo se mueve al mismo tiempo que se actualiza la red que lo genera), pero lo mantuvimos así deliberadamente para ser fieles a la versión de 2013, no a la de 2015.

DDQN (`double_dqn_agent.py`) — dos redes, con sync periódico duro

Acá sí hay dos redes explícitas: `model_a` (red online/policy) y `model_b` (red target). El manejo es:

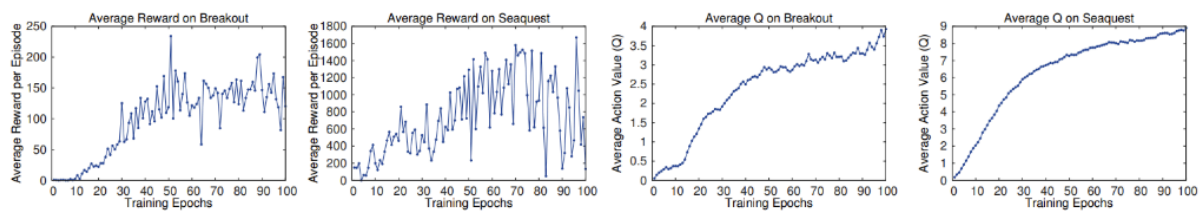
- **Selección de acción:** siempre con la red online (`model_a`), tanto para actuar en el entorno (ϵ -greedy) como para elegir cuál es la mejor acción del estado siguiente: $\arg\max_a Q_{\text{online}}(s', a)$.
- **Evaluación del target:** la acción elegida por la online se evalúa con la red target (`model_b`): $y = r + \gamma \cdot Q_{\text{target}}(s', \arg\max_a Q_{\text{online}}(s', a)) \cdot (1 - \text{terminated})$.
- **Sincronización:** la red target no se entrena por gradiente — se actualiza copiando los pesos de la red online cada `sync_target` pasos (parámetro del constructor, default 1000). Es una actualización dura/periódica (hard update, copia completa de pesos), no un promedio móvil tipo Polyak/soft-update.

Esta separación selección-vs-evaluación es exactamente el truco de van Hasselt et al. (2015) para reducir el sesgo de sobreestimación clásico de Q-learning: si la misma red elige y evalúa la mejor acción, tiende a sobreestimar sistemáticamente porque el max amplifica el ruido de estimación.

Por qué mantuvimos loss/optimizador iguales entre ambos

Aunque el paper de DDQN (2015) en rigor usa Huber loss + target network con la arquitectura de Nature 2015, decidimos mantener MSE + RMSProp en ambos agentes (igual que el DQN de 2013). La razón, explícita en el código: **así la comparación DQN vs. DDQN aísla únicamente el cambio de la regla del target** (y la presencia/ausencia de target network), sin mezclar además un cambio de función de loss y optimizador entre los dos experimentos — un control experimental más limpio para las conclusiones que pide la Parte 1 del enunciado (convergencia, estabilidad, exploración).

2. Durante la ejecución de algoritmos basados en el aprendizaje de la tabla Q, se puede observar una gran cantidad de ruido al graficar la reward obtenida por agentes utilizando políticas derivadas de distintos números de episodio a lo largo del proceso de aprendizaje. El trabajo del equipo de deep mind presenta una solución, qué se basa en medir Q en un conjunto arbitrario de estados y luego ir usándolos como canario a lo largo del proceso de aprendizaje. Las siguientes gráficas presentan los resultados de medición siguiendo ambas técnicas.



Discuta porque cree que se obtienen estos resultados.

Por qué el reward promedio por episodio es ruidoso

El propio paper lo explica en una frase clave: "small changes to the weights of a policy can lead to large changes in the distribution of states the policy visits". Hay varias razones que se combinan:

1. **Amplificación por retroalimentación (feedback loop) en la trayectoria:** el reward de un episodio depende de la secuencia completa de decisiones tomadas. Un cambio pequeño en los pesos de la red puede alterar una decisión temprana en el episodio (ej. en qué ángulo rebotar la pelota en Breakout, o cuándo subir a respirar in Seaquest), y ese cambio pequeño se propaga y compone a lo largo de todo el resto del episodio — llevando a trayectorias completamente distintas y, por lo tanto, a rewards muy distintos. Es un efecto "mariposa": ruido chico en los pesos

→ ruido grande en la trayectoria → ruido grande en el reward final.

2. El reward mide el rollout completo bajo una política estocástica: la evaluación se hace corriendo una política ϵ -greedy (con $\epsilon=0.05$ en el paper) durante un número fijo de pasos y promediando. Cada episodio individual introduce variabilidad extra por la exploración aleatoria en sí misma, además de la variabilidad propia del juego (posiciones iniciales de enemigos, etc. — muchos juegos de Atari tienen algo de aleatoriedad).

3. Es una señal "end-to-end", indirecta y de varianza acumulada: el reward por episodio es el resultado final de cientos de decisiones encadenadas, cada una con su propia fuente de ruido — todo ese ruido se acumula en un solo número por episodio, en vez de promediarse/cancelarse.

Por qué el Q promedio en un conjunto fijo de estados es mucho más suave

Acá está el detalle clave de la técnica que describe el enunciado: en vez de medir "qué tan bien juega la política ahora" (que depende de simular trayectorias completas y estocásticas), miden directamente cuánto vale, según la red, un conjunto fijo de estados — recolectado una sola vez con una política aleatoria antes de empezar a entrenar, y nunca vuelto a cambiar.

Esto elimina casi toda la fuente de ruido de la medición anterior:

- **No hay rollout de episodios:** no se simula ninguna trayectoria — es simplemente $Q(s, \cdot; \theta)$ evaluado sobre esos mismos estados fijos, en cada punto del entrenamiento. Es una función determinística de los pesos actuales, no una estadística sobre trayectorias estocásticas.

- La única variable que cambia entre mediciones son los pesos θ , que se actualizan de forma gradual e incremental vía SGD con un learning rate chico — por diseño, los pesos no pegan saltos bruscos de una medición a la siguiente, así que $Q(s, a; \theta)$ tampoco.

- Al ser siempre los mismos estados, no hay variabilidad extra por "a qué parte del juego llegó esta vez el agente" — es una comparación de manzanas con manzanas en cada checkpoint.

Por eso el paper la llama una métrica "más estable" para monitorear el progreso: sube de forma mucho más monótona porque refleja directamente la mejora acumulada del estimador de valor, sin la amplificación de simular episodios completos y estocásticos.

Sobre la diferencia entre Breakout y Seaquest

Vale la pena notarlo también como parte de la discusión: el reward de Seaquest es mucho más volátil (llega a picos de ~1600 y cae bruscamente incluso tarde en el entrenamiento, épocas 85-95) comparado con Breakout (oscila en una banda más contenida, ~100-200). Esto es consistente con la naturaleza del juego: en Seaquest el agente tiene que sostener una cadena más

larga y frágil de subtareas correctas (disparar enemigos, rescatar buzos, y crucialmente subir a respirar antes de quedarse sin oxígeno), donde un solo error puede terminar el episodio abruptamente (el submarino se queda sin aire) sin importar qué tan bien venía jugando el resto del episodio — mientras que en Breakout la dinámica es más autocontenida y determinística episodio a episodio. Esto agrega ruido adicional al reward de Seaquest más allá de lo que agrega la sensibilidad genérica política→trayectoria.

Conclusión (para conectar con lo que ya vieron de DQN vs DDQN)

Esta es exactamente la razón por la que, en la práctica, se prefiere monitorear el entrenamiento con la curva de Q promedio en un conjunto held-out en vez de (o además de) la curva de reward: da una señal de progreso mucho más legible y menos ruidosa. Ojo con un matiz que ya discutimos en la Parte 1 de su Obligatorio: como este Q-value es estimado por la propia red (sin target network en el DQN de 2013), una curva de Q que sube de forma consistente no garantiza que no haya sobreestimación — de hecho, la propia motivación de Double DQN (2015) parte de observar que estos valores de Q pueden inflarse sistemáticamente por encima de lo que el reward real justificaría.

3. ¿Qué función cumple la experience replay en las técnicas de Deep Q Learning?

En Deep Q-Learning, el experience replay cumple varias funciones que están directamente relacionadas entre sí, y en el Obligatorio quedan explícitas en **replay_memory.py**:

1. Rompe la correlación temporal entre muestras consecutivas

Si entrenamos la red con las transiciones en el orden en que ocurren (como en un update on-policy puro), cada minibatch estaría formado por estados casi idénticos entre sí (el auto se movió apenas un poco de un paso al siguiente). SGD asume implícitamente que las muestras son aproximadamente i.i.d.; entrenar con datos altamente correlacionados temporalmente hace que los gradientes estén sesgados hacia "lo que está pasando ahora mismo" y el entrenamiento se vuelve inestable/oscilante. Guardando transiciones en un buffer (ReplayMemory) y muestreando al azar (random.sample, uniforme, sin importar cuándo ocurrió cada transición) para armar cada minibatch, se rompe esa correlación: cada batch mezcla experiencia de distintos momentos y episodios.

2. Reutiliza cada transición varias veces (sample efficiency)

Sin replay, cada transición se usaría una sola vez para un único update y se descartaría — muy ineficiente, porque interactuar con el simulador (o el mundo real) es la parte cara. Con el buffer, cada transición queda disponible para ser muestreada muchas veces mientras dura en la memoria (capacidad BUFFER_SIZE = 10_000 en nuestro caso), exprimiendo mucho más aprendizaje de la misma cantidad de interacción con el entorno.

3. Estabiliza la distribución de entrenamiento

A medida que la política mejora, los estados que el agente visita cambian (el agente ya no pasa tanto tiempo en las zonas "malas" que aprendió a evitar). Si entrenáramos solo con experiencia reciente, la red iría "olvidando" progresivamente cómo comportarse en situaciones que ya no se visitan tanto (catastrophic forgetting), sesgándose hacia la experiencia más nueva. Mezclar en el buffer transiciones viejas y nuevas suaviza esa distribución cambiante y evita que la red sobreajuste al comportamiento reciente del agente.

Cómo lo manejamos concretamente en el Obligatorio

- **Buffer circular de capacidad fija** (BUFFER_SIZE = 10_000): al llenarse, la transición más antigua se sobrescribe (FIFO), así el buffer siempre contiene una ventana de experiencia relativamente reciente pero suficientemente variada.
- **Muestreo uniforme** (random.sample) de minibatches de tamaño BATCH_SIZE = 32 — todas las transiciones almacenadas tienen la misma probabilidad de ser elegidas (a diferencia de Prioritized Experience Replay, que muestrea con más probabilidad las transiciones con mayor error de TD; eso lo dejamos como mejora opcional para la Parte 3).
- **Warmup (learning_starts)**: no empezamos a entrenar hasta tener al menos max(batch_size, learning_starts) transiciones en el buffer, para no entrenar con un buffer casi vacío y poco diverso apenas arranca el entrenamiento.
- **Detalle de implementación por memoria**: guardamos state/next_state como uint8 (crudo, sin normalizar) y recién convertimos a float32 y mandamos a GPU al muestrear el minibatch — con observaciones de 4×84×336, guardar todo como float32 en el buffer sería inviable en RAM/VRAM.
- **Guardamos el flag terminated** (no done = terminated or truncated), para que el target de Bellman solo anule el bootstrap cuando el episodio terminó por un estado terminal real del MDP, y no por un corte de tiempo — esto no es parte de la función del replay en sí, pero es parte de cómo se maneja la tupla que se guarda en él.

1. ¿Qué problema de DQN soluciona Doble-DQN? ¿Qué observó en su obligatorio respecto a este punto?

El problema: la sobreestimación sistemática del valor Q (overestimation bias). En DQN vanilla, el target de Bellman es:

$$Y = R + \gamma \max_a Q(S', a')$$

usando la misma red tanto para elegir cuál es la mejor acción (argmax) como para evaluar su valor. Como las estimaciones de Q siempre tienen algo de ruido/error, tomar un max sobre estimaciones ruidosas tiende a sobreestimar sistemáticamente el valor real (es una consecuencia de la desigualdad de Jensen: el máximo de estimaciones ruidosas de un conjunto de valores tiende a superar el máximo de los valores reales). Ese sesgo se propaga y se amplifica episodio tras episodio a través del bootstrapping.

Cómo lo resuelve DDQN: desacopla selección de evaluación —

$$Y_t = R + \gamma Q_{\text{target}}(S', \operatorname{argmax}_a Q_{\text{online}}(S', a'))$$

la red online elige cuál es la mejor acción, pero la red target (una copia desfasada, sincronizada cada sync_target pasos) es la que evalúa cuánto vale esa acción. Como ambas redes rara vez están de acuerdo exactamente en cuál es "la mejor" acción, el sesgo de sobreestimación se reduce considerablemente.

Qué observar en su corrida (para cuando entrenen): la forma más directa de verificar esto empíricamente, tal como sugiere el enunciado del Obligatorio, es graficar el valor Q estimado a lo largo del entrenamiento para ambos agentes (por ejemplo, el $\max_a Q(s,a)$ promedio sobre un conjunto fijo de estados de validación). Deberían esperar ver que DQN reporta valores Q sistemáticamente más altos (a veces inflados de forma poco realista respecto al reward medio que efectivamente obtiene) mientras que DDQN da estimaciones más conservadoras y más consistentes con el reward real observado — y típicamente (aunque no siempre) una curva de reward más estable en DDQN. Si en su corrida no ven esa diferencia, puede ser señal de que el entrenamiento fue demasiado corto para que el sesgo se manifieste, o que sync_target está mal calibrado.

2. Vincule el uso de la replay memory con otras técnicas aprendidas en el curso.

La conexión más directa y explícita es con Dyna-Q (Laboratorio 6): ambas técnicas resuelven el mismo problema — sample efficiency, sacarle más provecho a cada paso real de interacción con el entorno — pero de formas distintas:

- **Dyna-Q** aprende un modelo del entorno a partir de la experiencia real ((pos, vel, action) → (next_pos, next_vel, reward)) y luego genera n pasos de "planning" adicionales por cada paso real, simulando transiciones con ese modelo aprendido para actualizar Q más veces sin necesitar más interacción real.
- **Experience replay en DQN** hace algo con el mismo objetivo pero sin aprender un modelo: guarda las transiciones reales ya vividas y las vuelve a muestrear (uniformemente al azar) para reentrenar sobre ellas múltiples veces.

En otras palabras: Dyna-Q "inventa" experiencia extra vía un modelo; el replay buffer "reutiliza" experiencia real ya recolectada. Ambas son formas de exprimir más aprendizaje de cada paso costoso de interacción con el entorno.

Otra conexión importante es con el carácter off-policy de Q-learning (visto en el lab de TD y el propio Dyna-Q): el replay buffer solo es seguro de usar porque Q-learning aprende sobre la política greedy independientemente de qué política generó los datos. Esto permite mezclar transiciones recolectadas bajo distintos valores históricos de epsilon sin invalidar el aprendizaje — algo que no sería tan directo con un algoritmo on-policy como Sarsa, que en principio debería aprender sobre la política que efectivamente generó los datos.

También hay un paralelo con la discusión de K-bandits sobre alpha constante vs. $1/N$ (no-estacionariedad): el buffer es circular y de tamaño acotado (BUFFER_SIZE = 10_000), descartando las transiciones más viejas. Igual que en el bandit no-estacionario se prefiere un alpha constante para no sobre-ponderar datos viejos que ya no representan la distribución actual, acá se evita que el agente siga entrenando indefinidamente sobre transiciones recolectadas bajo una política muy vieja (mucho más exploratoria) que ya no es representativa del comportamiento actual.

3. ¿Qué función resuelve la red neuronal del obligatorio? ¿Por qué es necesaria en la solución? ¿Por qué requiere una CNN?

Función: la red aproxima la función de valor $Q(s,a)$ — reemplaza a la Q -table de los labs anteriores (Q-learning tabular en MountainCar, FrozenLake, etc.).

Por qué es necesaria (y no alcanza con una tabla): en los labs previos discretizaban la observación a un puñado de estados (NUMBER_STATES × NUMBER_STATES) para poder indexar una tabla. Acá la observación es una imagen — un tensor (4, 84, 336) de píxeles — cuyo espacio de estados posibles es astronómico (cada combinación de valores de píxel es técnicamente un estado distinto). Es imposible enumerar o tabular eso; se necesita un aproximador de función que generalice: que dos imágenes parecidas (auto en carriles/posiciones similares) reciban valores Q similares, sin haber visto exactamente ese estado antes.

Por qué una CNN y no cualquier red densa: la entrada tiene estructura espacial (2D, con información local: bordes de carril, formas de vehículos, posiciones relativas). Una capa convolucional explota dos propiedades clave que un MLP sobre los píxeles crudos no aprovecha:

- **Localidad:** un filtro pequeño (ej. 8×8) mira una región acotada de la imagen, coherente con que las features relevantes (un auto, una línea de carril) son locales.
- **Invarianza a la traslación** (parcial): el mismo filtro se aplica en toda la imagen, así que un auto detectado en una zona se reconoce igual si aparece en otra zona — sin tener que aprender ese patrón por separado para cada posición posible.

Esto reduce drásticamente la cantidad de parámetros comparado con una red totalmente conectada directamente sobre los píxeles, y es exactamente el mismo argumento que usó Mnih et al. (2013) para procesar los frames de Atari.

4. Explique las dimensiones de la entrada de la red ¿Que significa $32 \times 4 \times 84 \times 84$?

Es un tensor de 4 dimensiones (batch_size, stack, height, width):

- **32: tamaño del batch** — cantidad de transiciones que se procesan juntas en un mismo forward/backward pass (viene del muestreo del replay buffer).
- **4: cantidad de frames apilados** (el stack temporal) — la dimensión que hace de "canales" de entrada a la primera Conv2d. No es color (RGB), es historia temporal: 4 observaciones consecutivas en escala de grises, necesarias para que la red pueda inferir velocidad/dirección por diferencia entre frames.
- **84×84 :** alto $\times$ ancho en píxeles del frame preprocesado (resize + grises), el tamaño clásico usado por Mnih et al. para Atari.

(Nota: en su implementación concreta sobre highway-env el tamaño real es (NUM_STACKED_FRAMES, OBS_HEIGHT, OBS_WIDTH) = (4, 84, 336) en vez de 84×84 , porque decidieron preservar el aspect ratio 4:1 de la autopista en vez de forzar un cuadrado — pero la lógica de las dimensiones es exactamente la misma.)

5. Si la entrada a la red es de tamaño $32 \times 4 \times 84 \times 84$, ¿de qué tamaño es la salida? ¿por qué?

La salida es (32, n_actions) — un vector de Q-values por cada elemento del batch, uno por cada acción posible.

Por qué: la red está diseñada para recibir un estado y devolver, en un único forward pass, el valor Q estimado para todas las acciones a la vez ($Q(s, a)$ para cada a), en vez de recibir (s,a) y devolver un escalar. Esto es clave para la eficiencia del algoritmo: para hacer $\arg\max_a Q(s,a)$ (selección de

acción) o $\max_a Q(s,a)$ (target de Bellman) solo hace falta un forward pass por estado, no uno por cada acción posible.

6. ¿Qué particularidad tiene la entrada de la parte FeedForward de la red implementada?

La parte "FeedForward" (las capas Linear/totalmente conectadas al final, después de las capas convolucionales) recibe un vector aplanado (`x.view(x.size(0), -1)`), no un tensor espacial. Es decir, la salida de la última capa convolucional (un tensor (batch, canales, h_out, w_out), todavía con estructura 2D) se colapsa a un vector 1D por cada elemento del batch antes de entrar a las capas densas, perdiendo deliberadamente la noción explícita de "posición" espacial de cada feature, y tratando cada valor del feature map como una entrada independiente.

La particularidad extra en este proyecto puntual: como el input preserva el aspect ratio rectangular de highway (336×84 , en vez del cuadrado 84×84 de Atari), el tamaño de ese vector aplanado (`h_out * w_out * out_channels`) es bastante más grande y asimétrico que en la arquitectura clásica de Atari — la primera capa Linear termina teniendo muchos más parámetros de entrada de los que tendría con la geometría cuadrada original del paper.

Posibles otras preguntas

squeeze / unsqueeze

`unsqueeze(dim)` — inserta una dimensión de tamaño 1 en la posición `dim`. `squeeze(dim)` — la elimina (solo si esa dimensión mide 1; si no le pasás `dim`, elimina todas las dimensiones de tamaño 1).

```
x = torch.zeros(5)           # (5,)
x.unsqueeze(0)                # (1, 5)  -> agrega dim al principio
x.unsqueeze(1)                # (5, 1)  -> agrega dim al final

y = torch.zeros(5, 1)
y.squeeze(1)                  # (5,)    -> la quita
y.squeeze(0)                  # (5, 1)  -> no hace nada, esa dim no mide 1
```

No mueven datos en memoria, solo reinterpretan la forma (son vistas, como `view`).

Acá el propósito es agregar la dimensión de batch: la CNN (`nn.Conv2d`) siempre espera (B, C, H, W). Si viene una sola observación (al elegir acción con `greedy_action`), es 3D y hay que fingir un batch de tamaño 1. Si ya viene un batch de `_sample_batch()`, es 4D y no se toca.

Función: `gather(dim, index)`

Qué hace: Selecciona, para cada fila del batch, el valor en la posición indicada por index

Dónde la usaron y por qué: `current_q_values.gather(1, actions_t.unsqueeze(1))`: de la Q de las 5 acciones por estado, se queda solo con el Q de la acción que realmente se tomó

Función: `argmax(dim=1)`

Qué hace: Índice del valor máximo a lo largo de esa dimensión

Dónde la usaron y por qué: En DDQN:

`policy_net_a(next_states_t).argmax(dim=1)` — la red online

elige cuál acción es mejor (esto es lo que diferencia a

Double DQN de DQN vanilla)

`torch.no_grad()`

Qué hace: Desactiva el cálculo de gradientes dentro del bloque

Dónde la usaron y por qué: Se usa para calcular el target de Bellman: el target no debe propagar gradiente, solo la Q actual (`current_q_values`) se entrena

`zero_grad() / backward() / step()`

Qué hace: Limpia gradientes acumulados / calcula gradiente de la loss / aplica el update con el optimizer

Dónde la usaron y por qué: El trío estándar de un paso de entrenamiento en PyTorch — sin `zero_grad()` los gradientes de steps anteriores se acumularían

`.item()`

Qué hace: Extrae un escalar Python de un tensor de 1 elemento

Resumen Entrega

0. Contexto y estructura de la entrega

El Obligatorio pide implementar y comparar DQN y Double DQN sobre highway-env, en tres partes:

- **Parte 1 — obligatoria.** Notebook principal (highway-fast-v0). Completar la implementación provista, entrenar DQN y DDQN hasta reward medio > 30 , graficar y sacar conclusiones, entregar video de cada agente.
- **Parte 2 — obligatoria.** Elegir un entorno de highway-env (merge-v0, roundabout-v0, parking-v0, intersection-v0 o racetrack-v0) y aplicar DQN o DDQN con objetivos y función de recompensa propios. Notebook más concisa: no re-explica conceptos ya cubiertos en la Parte 1.
- **Parte 3 — opcional.** Competencia de eficiencia: alcanzar reward medio ≥ 40 en highway-fast-v0 en el menor tiempo de entrenamiento posible, con DQN/DDQN o variantes directas (Rainbow, Dueling, PER, vectorización).

Resumen de qué se entregó en este repo, de un vistazo:

Parte	Entorno	Técnica	Resultado	¿Cumple el mínimo pedido?
1	highway-fast-v0	DQN	reward medio final ≈ 59.6	Sí (> 30)
1	highway-fast-v0	DDQN	reward medio final ≈ 55.0	Sí (> 30)
2	merge-v0	DDQN	reward 11.65 · colisión 6.0%	Sí (objetivo propio ≥ 10 / $< 15\%$)
3 (opc.)	highway-fast-v0	DDQN + PER + vectorización	eval 63.38 ± 11.22 en 3.77 min	Sí (≥ 40)

1. Parte 1 — DQN vs Double DQN en highway-fast-v0

1.1 Arquitectura del proyecto

La notebook orquesta todo; delega la lógica a módulos .py separados, cada uno con una responsabilidad:

- **replay_memory.py** — cola circular de transiciones (Transition = state, action, reward, terminated, next_state).
- **abstract_agent.py** — clase base Agent: train(), select_action() (ϵ -greedy), greedy_action(), play(), _sample_batch(). Todo lo común a DQN y DDQN vive acá.
- **dqn_agent.py** — subclase de Agent; solo redefine update_weights() con el target de Bellman calculado con UNA sola red.
- **double_dqn_agent.py** — subclase de Agent; redefine update_weights() con dos redes (online elige, target evalúa).
- **dqn_cnn_model.py** — CNN que aproxima $Q(s, \cdot)$: 2 capas convolucionales + 2 capas densas, arquitectura simplificada respecto al paper completo de Mnih.
- **utils.py** — make_env(): construye el entorno con el pipeline de wrappers de imagen (render→resize→grises→stacking) y process_state() (ϕ), que normaliza a [0,1] y agrega la dimensión de batch.

Decisión central del diseño: DQNAgent y DoubleDQNAgent son subclases de la misma clase base y solo difieren en cómo calculan el target de Bellman dentro de update_weights(). Todo el resto del loop de entrenamiento (recolección de experiencia, ϵ -greedy, guardado en el buffer, muestreo) es idéntico — esto es intencional, para que la comparación DQN vs DDQN aísle únicamente el cambio algorítmico.

1.2 Por qué observación visual y no la KinematicObservation nativa

highway-env entrega por defecto una KinematicObservation (posición/velocidad de vehículos cercanos en forma tabular) — un "oráculo simbólico" que resolvería el problema con un MLP chico, sin que el agente aprenda percepción. El Obligatorio fuerza entrada visual (wrapper _RenderAsObservation) para replicar el régimen del paper de Mnih et al. (2013): la red tiene que aprender features visuales (carriles, vehículos) además de la política, igual que en Atari.

Pipeline de preprocesamiento (ϕ), aplicado en este orden en la entrega final:

Paso	Transformación	Resultado	Por qué
1	Render como observación	Frame RGB 600×150	Reemplaza la KinematicObservation por la imagen del entorno
2	Resize	336×84 (aspecto 4:1)	highway es horizontal (4:1), no cuadrado como Atari; forzar 84×84 distorsionaría la geometría de carriles
3	Escala de grises (BT.601)	336×84 × 1 canal	El color no aporta info útil; reduce el input 3×
4	Frame stacking (4 frames)	(4, 84, 336) uint8	Un frame no tiene velocidad; 4 frames permiten inferirla por diferencia

El tensor final (4, 84, 336) se guarda en el buffer como uint8 (no float32): con un buffer de 10.000 transiciones, guardar en float32 requeriría ≈45 GB; en uint8 son ≈11 GB. La conversión a float32 en [0,1] (dividiendo por 255) ocurre recién al muestrear el minibatch para entrenar, en `process_state()`.

1.3 Hiperparámetros finales usados en el entrenamiento

Hiperparámetro	Valor	Qué controla
TOTAL_STEPS	300.000	Tope global de pasos de entrenamiento
EPISODES (tope)	5.000	Corta antes si se llega a TOTAL_STEPS
STEPS_PER_EPISODE	150	Límite de pasos por episodio
GAMMA (γ)	0.99	Factor de descuento de recompensas futuras
EPSILON_INI / EPSILON_MIN	1.0 / 0.05	Exploración inicial 100% → mínimo 5% (nunca 0)
EPSILON_ANNEAL_STEPS	20.000	Pasos para bajar ε de 1.0 a 0.05 (lineal)
BATCH_SIZE	32	Transiciones por actualización
BUFFER_SIZE	10.000	Capacidad del replay buffer
LEARNING_RATE	5e-5	Tasa de aprendizaje de RMSProp
LEARNING_STARTS	1.000	Pasos de warmup antes de entrenar

SYNC_TARGET (solo DDQN)	1.000	Cada cuántos pasos se copia la red online → target
max_grad_norm	10.0	Gradient clipping (desviación deliberada del paper 2013, ver 1.5)
HIGH_SPEED_REWARD	0.8 (default 0.4)	Duplica el valor de ir rápido, para incentivar adelantar
VEHICLES_COUNT	30 (default 20)	Más denso / desafiante
OBS_HEIGHT × OBS_WIDTH	84 × 336	Resize preservando aspecto 4:1 de highway
SIM_FREQUENCY train / eval	5 / 30	Frame-skip: 5 Hz para entrenar rápido, 30 Hz para video fluido

Arquitectura final de la CNN (DQN_CNN_Model, confirmada con `torchinfo.summary()`):
 Conv1(in=4, out=16, kernel=8, stride=4) → Conv2(in=16, out=32, kernel=4, stride=2) → flatten
 (32×9×40 = 11.520) → FC1(11.520→256) → FC2(256→5 acciones). Total: 2.962.997 parámetros.

Es una versión reducida de la CNN completa del paper (2 capas conv en vez de 3, FC de 256 en vez de 512), justificada por las restricciones de tiempo/cómputo de un entorno de curso.

1.4 Resultados obtenidos

Ambos agentes superan el mínimo pedido por la consigna (reward medio > 30):

Agente	Reward medio final (ventana 50 ep.)	Tiempo de entrenamiento	5000 episodios
DQN	59.6	2 h 36 min (≈ 1.88 s/episodio)	100%
DDQN	55.0	1 h 27 min (≈ 1.05 s/episodio)	100%

Además de la corrida base, se hizo una búsqueda de hiperparámetros con Optuna (TPESampler, seed=42, 6 trials de 1000 episodios cada uno) sobre gamma (0.90–0.99), learning_rate ($1e-5$ – $1e-3$), epsilon_anneal_steps (5k–30k), epsilon_f (0.01–0.10) y learning_starts (500–5000). Los dos mejores trials se re-entrenaron a 5000 episodios completos:

Trial	gamma	lr	eps_anneal	eps_f	learning_starts	Reward (1000 ep.)	Reward final (5000 ep.)
5 (mejor)	0.9707	$2.51e-5$	20.000	0.0633	500	48.10	58.4 (98% completado, cortado por tiempo)
4	0.9551	$1.9e-5$	10.000	0.0430	2.500	29.65	32.6
0 / 1 / 2 / 3	—	—	—	—	—	7.9 – 8.7	no se re-entrenaron a full

1.5 Decisiones de diseño que se desvían deliberadamente del paper

- **Gradient clipping** — aplicado con max_grad_norm=10.0. El paper original de 2013 no lo usa. Se agregó porque, sin clipping, la gráfica de Q promedio mostraba ~5 picos grandes de ruido distribuidos en el entrenamiento; con clipping esos picos se atenúan.
- **Resize** — 336×84 en vez de 84×84 (Atari). Preserva el aspect ratio 4:1 real del render de highway (autopista horizontal); forzar un cuadrado distorsionaría la geometría de carriles/vehículos, a costa de una primera capa densa más grande.
- **Frame-skip** — en vez de un wrapper AtariPreprocessing(frame_skip=k), el mismo efecto se logra ajustando simulation_frequency (5 en train, 30 en eval) sobre policy_frequency=1 — algorítmicamente idéntico, pero implementado a nivel de configuración del simulador.

- **Reward shaping / densidad de tráfico** — HIGH_SPEED_REWARD subido a 0.8 (default 0.4) para romper el óptimo local de "igualar la velocidad del vecino" en vez de adelantar; VEHICLES_COUNT subido a 30 para un entorno más desafiante.
- **Elección de entorno** — se usa highway-fast-v0 en vez de highway-v0 por rendimiento: evita chequeos de colisión $O(N^2)$ entre todos los NPCs.

2. Parte 2 — merge-v0 con Double DQN

2.1 Entorno elegido y justificación

El equipo eligió **merge-v0**: el vehículo ego debe incorporarse desde un carril de entrada al carril principal de la autopista, donde los vehículos que ya circulan mantienen su velocidad y no ceden el paso automáticamente. Espacio de acciones idéntico al de highway-fast-v0 (Discrete(5): LANE_LEFT, IDLE, LANE_RIGHT, FASTER, SLOWER).

Por qué DDQN (y no DQN) para este entorno en particular, según lo que escribió el equipo:

“en entornos con acciones de alto impacto como el merge, DQN tiende a sobreestimar los Q-values de acciones agresivas (FASTER, LANE_LEFT). DDQN no sobrestima en la selección de acciones en evaluación, esto reduce sesgo y produce una política más conservadora y segura.”

2.2 Objetivos propios y criterio de éxito

Antes de entrenar, midieron una línea de base con agente aleatorio (100 episodios): reward medio ≈ 7.0 , tasa de colisión $\approx 77\text{--}83\%$. A partir de eso definieron objetivos propios:

Métrica	Baseline (aleatorio)	Objetivo definido
Reward medio / episodio	≈ 7.00	≥ 10.0
Tasa de colisión	$\approx 77\text{--}83\%$	$< 15\%$
"Merge-quality score"	≈ 0.10	> 0.60

Cambios de hiperparámetros respecto a la Parte 1 (misma arquitectura de red, sin modificar DQN_CNN_Model): STEPS_PER_EPISODE=25 (los episodios de merge son muy cortos, ~ 10 pasos), BUFFER_SIZE=2.000 (mucho menor, coherente con episodios cortos), EPSILON_ANNEAL_STEPS=10.000, SYNC_TARGET=500, HIGH_SPEED_REWARD=0.8.

2.3 Resultados y conclusiones del equipo

Evaluación final (50 episodios greedy, política ya entrenada):

Métrica	Resultado	Objetivo	¿Cumplido?
Reward medio	11.650	≥ 10.0	Sí
Tasa de colisión	6.0%	$< 15\%$	Sí

Conclusiones citadas textualmente de la notebook de Parte 2:

“el reward de entrenamiento (~11) es levemente inferior al reward de evaluación (~12.2). La diferencia se explica por el epsilon residual de 0.05 durante training.”

“Los Q-values crecen de forma sostenida durante las primeras ~10.000 episodios y luego se estabilizan... La poca variabilidad de la curva puede indicar que DDQN redujo efectivamente el sesgo de sobreestimación presente en DQN vanilla.”

“Los episodios se vuelven progresivamente más largos... pasando de ~8 a más de 13 pasos en promedio.”

“El agente superó ampliamente los objetivos de reward y colisión.”

3. Parte 3 — Desafío de eficiencia (opcional)

3.1 Objetivo y resultado final

Objetivo de la consigna: reward medio ≥ 40 en highway-fast-v0, en el menor tiempo de entrenamiento posible. Resultado obtenido por el equipo: eval_mean = 63.38 ± 11.22 (sobre 100 episodios greedy), en 3.77 minutos de entrenamiento (corte por early stopping), tras 2.120 episodios, sobre GPU NVIDIA RTX 5060 Ti local (no Colab). El propio reporte JSON generado por la notebook confirma "passed": true.

3.2 Técnicas de eficiencia implementadas

- **Prioritized Experience Replay (prioritized_replay_memory.py)** — Sum-tree proporcional (Schaul et al., 2016). Prioridad = $(|\text{error TD}| + \epsilon)^\alpha$ con $\alpha=0.6$. Muestreo estratificado por segmentos del sum-tree. Importance-sampling weights normalizados por el máximo del batch, con β annealed linealmente de 0.4 a 1.0 en 100.000 steps. Las transiciones nuevas entran con la prioridad máxima vista hasta el momento.
- **Vectorización de entornos (vector_utils.py)** — gymnasium.vector.AsyncVectorEnv con context="spawn" explícito (evita cuelgues de pygame/SDL en Colab/Linux) y SDL_VIDEODRIVER=dummy fijado antes del spawn. N_ENVS = min(11, cpu_count) en la corrida final. La selección ϵ -greedy se vectoriza en un único forward pass de tamaño N.

- **Mayor replay ratio (grad_steps_per_iter)** — de 1 a 4 actualizaciones de gradiente por cada iteración de rollout (aprovecha mejor cada paso de interacción real, en la misma línea que el motivo original del replay buffer).
- **Reducción de resolución de render** — reducción de la resolución nativa del render de 600×150 a 240×120 antes de aplicar el resize a 84×336 — un parámetro nuevo (screen_width/screen_height) agregado a make_env() en utils_3.py, distinto del resize final.
- **Reorden grayscale → resize** — se invirtió el orden respecto a la Parte 1 (antes: resize→grises; ahora: grises→resize), para abaratar el resize operando sobre 1 canal en vez de 3.
- **Corte por wall-time + early stopping** — el entrenamiento corta por tiempo real (max_wall_time_min, cap duro de 30 min para que la re-ejecución del profesor no cuelgue) en vez de por episodios/steps fijos, con early stopping si el reward móvil sostiene el umbral objetivo durante varios chequeos consecutivos.
- **Double DQN como base** — fast_dqn_agent.py reutiliza la regla de target de double_dqn_agent.py (acción elegida con la red online, evaluada con la red target) — declarado explícitamente como reutilización, no como técnica nueva de esta parte.

Importante para la defensa: la Parte 3 no implementa Rainbow, Dueling DQN ni n-step returns, pese a que la celda de encabezado de la notebook los menciona como opciones (quedó marcada como "Mejoras aplicadas: TODO" y no se actualizó). La combinación real y efectivamente usada es: Double DQN + Prioritized Experience Replay + rollout vectorizado (AsyncVectorEnv) + early stopping + los hiperparámetros óptimos hallados con Optuna en la Parte 1.

abstract_agent_3.py y utils_3.py son, en la práctica, copias casi idénticas de los archivos de la Parte 1 (diferencias solo cosméticas), salvo dos cambios funcionales reales: la exposición de screen_width/screen_height y el reorden grayscale→resize. Se mantuvieron como archivos separados a propósito, para no tocar los archivos que son la entrega calificada de la Parte 1 (abstract_agent.py, dqn_agent.py, double_dqn_agent.py).

3.3 Progresión de mejoras (texto citado de la notebook)

Mejora 1 — vectorización con N_ENVS=8:

“el reward (últimos 50) supera el target de 40 de forma esporádica desde el minuto ~4.4, pero recién se estabiliza por encima de 40... cerca del minuto ~11-12... El resultado final, tras los 30 min, fue de un reward 63.38.”

Mejora 2 — replay ratio 4x + buffer 50k + render 240×120:

“con los tres cambios combinados, el reward... se estabiliza... hacia el minuto ~3... una mejora de ~4x en tiempo hasta converger. El throughput bruto bajó levemente (~151 → ~134 steps/seg)... un trade-off esperado.”

Mejora 3 — hiperparámetros del Trial 5 de Optuna + N_ENVS=11 + reorden grises/resize:

“el tiempo hasta el early stop bajó de 8.67 min... a 3.77 min, con... el mismo reward final (63.38 ± 11.22 en ambos casos).”

Limitación que el propio equipo dejó anotada sin resolver: ϵ -anneal_steps=20.000 podría acortarse más en este escenario, ya que con N_ENVS=11 esos mismos 20.000 steps se acumulan en poco más de 2 minutos de wall-time (el anneal de exploración ya no está calibrado para la nueva velocidad de recolección de datos que da la vectorización).

4. Glosario rápido

Término	Definición breve
MDP	Markov Decision Process: (S, A, P, R, γ). El agente está en s, toma a, recibe r, pasa a s'.
Q(s,a)	Recompensa acumulada esperada al tomar a en s y luego seguir la política óptima.
Ecuación de Bellman	$Q^*(s,a) = E[r + \gamma \cdot \max_{a'} Q^*(s',a')]$. Relación recursiva del Q óptimo.
ϵ-greedy	Con prob. ϵ : acción aleatoria (explorar). Con prob. $1-\epsilon$: $\arg\max Q(s,a)$ (explotar).
Replay buffer	Buffer circular de transiciones pasadas; rompe correlación temporal, reutiliza experiencia.
Target network	Copia de la red online, sincronizada periódicamente; estabiliza el target de Bellman.
Sobreestimación (DQN)	El max sobre Q-values ruidosos tiende a inflar el valor estimado (misma red elige y evalúa).
Double DQN	Desacopla selección (red online) de evaluación (red target) para reducir la sobreestimación.
terminated vs truncated	terminated: fin real del MDP. truncated: corte por timeout. Solo terminated anula el bootstrap.
Frame stacking	Apilar los últimos k frames como input, para poder inferir velocidad/aceleración.
PER	Prioritized Experience Replay: muestrea con más probabilidad transiciones con mayor error TD.
Vectorización	Correr N entornos en paralelo (AsyncVectorEnv) para recolectar experiencia más rápido.
Wall-time	Tiempo real de reloj transcurrido (vs. cantidad de episodios/steps), usado como criterio de corte en Parte 3.